As características de uma API são:

Ter um protocolo de comunicação: Um protocolo capaz de reagir a requests e responses. **(HTTP, HTTP/3, HTTPS, SMTP, Web Sockets e etc.)**

Ser um servidor: Uma aplicação capaz de servir, entregar, gerar algum conteúdo(content). **Sempre seguindo os padrões do protocolo adotado.**

Padrões de arquitetura (Services ou use case): veja abaixo os principais padrões usados no mercado hoje:

- **REST (Representational State Transfer):** É o padrão mais popular para APIs web. Utiliza o protocolo HTTP (com métodos como `GET`, `POST`, `PUT`, `PATCH` e `DELETE`), opera de forma **sem estado** (*stateless*) e costuma retornar dados em formatos leves como **JSON** ou XML. [1, 2, 3, 4, 5, 6]
- **SOAP (Simple Object Access Protocol):** Um protocolo mais rígido e baseado em padrões corporativos antigos, que exige mensagens estritamente formatadas em **XML**. Oferece alto nível de segurança e controle de transações, sendo muito usado em sistemas bancários e legados. [1]
- **GraphQL:** Desenvolvido pelo Facebook, permite que o cliente especifique exatamente quais **dados precisa** em uma única requisição, evitando o problema de *over-fetching* (receber dados demais) ou *under-fetching* (receber dados de menos) comum no REST. [1]
- **gRPC:** Criado pelo Google, utiliza **HTTP/2** e o **Protocol Buffers** (Protobuf) para serialização binária. É extremamente rápido, de baixo consumo de recursos e muito utilizado para comunicação interna de alta performance entre **microsserviços**. [1, 2]
- **RPC (Remote Procedure Call):** Um modelo baseado na execução de **procedimentos ou funções remotas**, onde a chamada se parece com a execução de uma função local no código, com menor foco em recursos e estados (exemplos incluem XML-RPC e JSON-RPC). [1]
- **APIs Orientadas a Eventos (Event-Driven):** Em vez do modelo tradicional de requisição e resposta, a comunicação é baseada na **publicação e consumo de**

eventos (frequentemente integradas a brokers como Kafka ou RabbitMQ), ideais para sistemas reativos e distribuídos. [1, 2]

- **WebSockets:** Permite uma **conexão bidirecional em tempo real** e de longa duração entre cliente e servidor, muito usada em chats, placares ao vivo e aplicações colaborativas. [1]
- **Webhooks:** Conhecidos como "APIs reversas", funcionam por meio de notificações onde o servidor envia um alerta HTTP para uma URL do cliente assim que um **evento específico acontece**. [1]

Você pode saber mais sobre os princípios de construção no artigo da IBM sobre API design ou consultar boas práticas de implementação na Documentação da Microsoft Azure sobre Design de API Web RESTful. [1, 2]

Dentro desses padrões, iremos focar principalmente no REST e RESTful.